

Skill Loom: Evidence-Bounded Maintenance of Agent Skill Libraries

Skill Loom contributors
6 October 2026

Systems technical report · version 0.1 · not peer reviewed

Abstract

Agent skills package reusable instructions, scripts and resources, but a growing library introduces a maintenance problem distinct from generating a useful answer. Candidate quality, user fit, host compatibility, deployment integrity and downstream utility require different evidence. We describe Skill Loom, a local toolkit that separates these concerns through explicit user profiles, bounded source discovery, capability-gap analysis, reviewed file-change plans, recoverable journals and task-bound evidence receipts. The design is grounded in a focused review of fifteen primary papers and official engineering sources; it integrates established ideas rather than claiming a new learning algorithm. We evaluate twelve deliberately constructed software-contract scenarios and a real maintenance cycle in one Windows Codex environment containing 39 user-maintained skills. The reviewed transaction implementation satisfies all eight selected mutation contracts; bound receipts satisfy all four selected evidence contracts. These results establish only the tested software properties. They do not demonstrate higher agent task accuracy, reduced token expenditure, general security, or universal compatibility. We provide reproducible code, failure cases, an anonymized skill catalog and a separate plan for future paired model-task evaluations.

Keywords: agent skills; lifecycle management; provenance; evaluation; reversible deployment; human-agent collaboration.

1. Introduction

File-based agent skills expose reusable procedural knowledge through metadata, instructions and optional resources. Progressive disclosure allows a host to inspect descriptions before loading relevant content [1]. However, packaging and discoverability do not establish usefulness. SkillsBench evaluates task performance under controlled skill conditions and reports heterogeneous effects [2]. A lifecycle survey further distinguishes proposal, admission, retrieval, maintenance and governance as separate concerns [3].

The practical problem addressed here is maintaining a personal skill library under changing tasks, users, models and harnesses. A candidate can be relevant but unlicensed for the intended redistribution, syntactically valid but behaviorally ineffective, or useful yet deployed incorrectly. A rollback can restore bytes while losing executable permissions. A receipt can reference unchanged evidence from the wrong task. These are distinguishable failure modes, and a single quality score cannot resolve all of them.

This report asks two bounded questions. **RQ1:** can explicit change plans and recoverable journals preserve specified mutation and rollback contracts under selected faults? **RQ2:** can task, candidate and time binding reject selected stale or replayed evidence receipts? A third question -- whether the full workflow improves agent task quality and cost across users and harnesses -- remains open. The contribution is a reproducible engineering artifact, its operational contracts, a focused synthesis of prior work and a small fault-injection study. We do not claim priority for the lifecycle, reflection, deterministic-host or rollback concepts.

2. Related work and design implications

2.1 Experience, reflection and candidate search

Voyager stores reusable executable skills and refines programs using environmental feedback [4]. Reflexion uses verbal feedback as persistent episodic information, while Self-Refine iterates an output through feedback and revision [5], [6]. These mechanisms motivate reusable diagnosis, but they do not imply that every reflection should become a global instruction.

GEPA searches prompt candidates with reflection and validation-informed selection [7]. EvoSkill applies a proposer/builder organization to skill discovery [8]. SkillRL jointly changes policies and skill libraries through training [9]. AutoSkill addresses experience-based skill extraction and personalized library maintenance [10]. ACE emphasizes incremental context updates and structured curation [11]. Skill Loom borrows the separation between proposal and selection; its manually graded ranking queue is neither a reproduction of these optimizers nor a trained utility predictor.

2.2 Verification, compression and negative evidence

CoEvoSkills uses co-evolving verification while retaining oracle feedback within its reported procedure [12]. SkillZip already describes structural preservation, provenance and deterministic host-managed writes [13]. These are direct precedents for important aspects of our engineering design. In both cases, verification assumptions matter: a surrogate or a parsed structural contract is not unrestricted semantic correctness.

Rethinking Self-Evolving Agent Skills studies multi-round feedback and reports model- and task-dependent returns rather than guaranteed monotonic improvement [14]. SkillCoach separates skill-use process evaluation from final outcomes [15]. A broader evaluation survey organizes mechanisms and benchmark coverage [16]. Together, these works support preserving rejected candidates, no-change outcomes and honest stopping states. They also argue against using installation count or self-reported success as the optimization target.

2.3 Operational maintenance

The technical-debt perspective highlights hidden dependencies and feedback loops in maintained learning systems [17]. We use this as an engineering analogy, not as evidence about SKILL.md performance. Official agent-evaluation guidance distinguishes tasks, trials, transcripts, outcomes and graders [18]. Context-engineering guidance motivates selective retrieval and clear tool responsibilities [19]. Accordingly, Skill Loom separates static inspection, host discovery, script checks and task-level evaluation.

The accompanying literature matrix fixes paper versions and gives section-level evidence locations. It notes incompatible benchmark versions, limited replication and reported inconsistencies instead of constructing a cross-paper leaderboard. None of the cited model experiments was reproduced in this study.

3. Problem formulation

Let an execution setting be $E=(U,M,H,T,P,D,V,B)$, comprising a user profile, model, harness, tools, permissions, task distribution, evaluator and budget. A library state L_t includes skill content, source identities, local adaptations and deployment metadata. A proposal mechanism produces a candidate C_t from selected task evidence. Admission either retains the existing library or applies a reviewed transition:

$$L_{t+1} = A(L_t, C_t, E).$$

Retention is a legitimate result. Three properties must be distinguished: **proposal relevance**, **state-transition correctness**, and **behavioral utility**. A verified file-state transition does not establish proposal relevance or behavioral utility, and a useful candidate does not excuse an incorrect state transition.

For a task set D , one may seek a portfolio balancing quality Q , cost C and maintenance burden K :

$$\max_L Q(L; E, D) - \lambda C(L; E, D) - \mu K(L),$$

subject to user authorization, output requirements and compatibility constraints. This expression defines a design objective, not an implemented optimizer or a proven convergence result. The prototype's profile weights order an investigation queue; unknown evidence remains unknown and hard adoption conditions are reported separately.

3.1 Transition contracts and assumptions

A reviewed plan contains before/after file hashes, supported permission modes, exact changes, a physical target root and a disjoint journal location. Application rechecks both snapshots, including a no-change plan. Modified originals are backed up before writes. A completed transition must match its after state. An interrupted transition retains a journal for recovery. Rollback refuses later user changes rather than overwriting them.

Under cooperative exclusive writing, available storage and successful filesystem operations, these checks provide a direct argument for the tested byte-and-mode postconditions: each planned path receives the reviewed content and supported mode, and the final snapshot is compared with the plan. This is not a proof of crash-consistent multi-file atomicity. ACLs, extended attributes, directory metadata, adversarial same-user races and arbitrary storage failures are outside the claim. Individual file replacement is used; a multi-file transition may be interrupted and require explicit recovery.

4. System design

4.1 User needs and bounded discovery

An editable private profile records required capabilities and user-selected budgets. A lightweight environment check reports OS, Python and executable presence; it does not scan credentials or infer preferences. Capability observations distinguish skill, environment, source and unknown causes. Existing providers are considered before searching or creating a skill. An unobserved skill is not automatically removed.

Source discovery observes registered repositories and optionally searches explicit public keywords. Coverage is the registered source set and bounded search sample, not the entire internet. Staging requires a full commit, verifies downloaded Git blob identities and lengths, retains source hashes and license files, and does not execute downloaded code. Five evidence-backed dimensions support candidate review: task fit, evidence, maintainability, efficiency and portability. These dimensions are configurable policy, not calibrated probabilities.

4.2 Portfolio and deployment

The catalog maps skills to capability identifiers and scope. Capability overlap and user-defined metadata budgets identify review candidates. Similarity does not establish redundancy: two skills may serve different artifacts or authorization boundaries. Merging, narrowing, retiring or keeping an item requires examining representative tasks and its consumers.

Deployment is a separate command from discovery and ranking. The plan digest selects the reviewed plan; it is an integrity identifier, not a signature or grant of authority. Journals reside outside the

discovery root. Discovery can inspect a linked entry, whereas mutation requires the reviewed physical skills root. The prototype does not change host approval settings or silently install hooks.

4.3 Evidence receipts and bounded repair

Receipts enumerate required checks and reference evidence bytes. Optional CLI constraints bind a receipt to a run identifier, candidate digest and age limit; the supplied Stop-hook example requires run and candidate bindings. A failed first Stop may request one scoped repair. A repeated Stop or an unavailable gate permits termination while communicating an unverified state. Allowing termination never converts failure to success.

The receipt checker verifies declarations and artifacts, not their truthfulness. A fabricated report with matching hashes remains a fabricated report. Grader correctness, real command execution and the independence of an expected candidate identifier require external controls and review.

4.4 Portability and data management

The common layer is file-based and independent of any model API. Host-specific discovery paths, hook semantics and delegation remain explicit adapters. Missing subagent support falls back to serial work; unavailable hooks fall back to explicit gate execution. Protocol compatibility and behavioral transfer are separate evaluation targets.

Raw traces, private profiles and journals remain outside the public repository. The optional trace observer emits structural counts without prompts, commands or tool outputs. Cache cleanup accepts only explicitly adopted reconstructible directories, rechecks exact files and records deletion intent before unlinking. It does not manage credentials, sessions, databases or plugin caches.

5. Evaluation protocol

5.1 Deterministic fault injection

The study uses twelve hand-selected scenarios, each in a fresh temporary directory. Eight mutation scenarios cover a clean update, missing reference, Python syntax error, candidate drift, runtime drift, an added local file, post-install rollback drift and a corrupt backup. Four receipt scenarios cover a valid record, changed artifact, another task's receipt and an expired receipt.

The comparators are small reference implementations: unchecked copying, static inspection followed by copying, and declared-pass receipt acceptance. They are intentionally weak operational baselines, not published skill-evolution systems or production package managers. The tested contract determines whether application/acceptance or refusal is expected. Source code and per-case outcomes are provided in `scripts/benchmark_invariants.py` and `research/results/invariants.json`.

Scenarios are the descriptive units. Repeated deterministic executions are not independent samples; no confidence interval, significance test or population success probability is estimated. The cases were chosen during development and therefore do not constitute an independent held-out benchmark. The study tests mechanism behavior, not generalization.

5.2 Local deployment study

A single Windows environment with Python 3.12.8 and Codex CLI 0.160.1 contained 39 user-maintained skills. We inventoried them and applied two reviewed file changes to its existing maintenance skill, adding an on-demand reference to the new workflow. We then restored the original state, compared it with the pre-change snapshot, reapplied the candidate and checked a no-change run. Global host configuration and instruction-file hashes were compared before and after. This is a case study of deployment mechanics, not a representative sample of users.

6. Results

Contract family	Reference implementation	Satisfied / selected cases
File mutation and rollback	Unchecked copy	1 / 8
File mutation and rollback	Static check plus copy	3 / 8
File mutation and rollback	Reviewed transaction	8 / 8
Receipt acceptance	Declared pass only	1 / 4
Receipt acceptance	Task/candidate/time-bound receipt	4 / 4

The static comparator rejects missing references and syntax errors but does not enforce reviewed-state or rollback conditions. The transaction implementation satisfies the eight selected contracts. Bound receipts reject the three selected stale/replay cases while accepting the valid record. These differences are expected consequences of the implemented checks and should not be interpreted as broad empirical superiority.

In the local deployment cycle, the skill count remained 39; the final static inventory reported zero errors and zero warnings. Rollback matched the original file snapshot, reapplication completed, and the no-change run returned without changing content. The protected host configuration hashes remained unchanged. No new same-purpose maintenance skill was added to the global discovery catalog.

Independent review exposed additional implementation gaps: executable permission loss on POSIX, missing drift checks on no-change plans, incomplete cleanup interruption records, weak input-type checks, missing Git blob verification, nonfinite freshness limits and a case-insensitive reserved-name collision. The implementation and regression tests were amended. The latest software-test and CI status is separately recorded in docs/validation.md; platform-skipped tests are not counted as executed checks. Native Claude Code hook operation and broad model-task transfer remain unverified in this local study.

7. Limitations and next experiments

The project does not establish an automatic quality oracle, worldwide discovery coverage or downstream model improvement. Its source ranking depends on reviewed annotations. Capability maps can be incomplete. Read-only syntax checks do not detect arbitrary malicious code. Hashes do not validate semantics. The filesystem protocol assumes cooperative writers and does not preserve every OS-specific metadata field. The scope is a local prototype rather than a multi-tenant managed service.

A subsequent behavioral study should compare no skill, the current stable skill and the candidate on matched tasks, fixing model, harness, tools and budget. Selection tasks must be separate from frozen evaluation tasks. Process measures should include correct and spurious skill selection, while outcome measures should include artifact fidelity, success, retries, latency and total cost. A multi-round study should count rejected candidates, no-ops and retained old versions, and compare equal-budget alternatives. Model or user migration requires new tests rather than inherited confidence.

These experiments are proposed, not completed. Larger sample sizes should follow an explicit research question, a variance estimate and a feasible budget; this report does not invent expected improvements or a power calculation.

8. Conclusion

Skill Loom makes a limited but useful distinction operational: generating a candidate, changing a library correctly and improving a user's tasks are different achievements. Its local toolkit supports inspectable decisions, bounded repair and reversible deployment. The reported fault-injection and local case-study results support specified software contracts, while leaving model utility, broad portability and comparative usability as explicit future evaluation questions.

Availability and disclosure

Code, anonymized examples, source records, experiment scripts and this manuscript are released with the project. Third-party skill content is not redistributed by the catalog. Development used an AI coding assistant and independent agent reviews; the promotional card is AI-generated. The technical report has not undergone academic peer review and does not imply acceptance by a venue. No private user transcripts or credentials are included in the public artifact.

References

- [1] B. Zhang, K. Lazuka, and M. Murag. Equipping agents for the real world with Agent Skills. Anthropic Engineering, 2025. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
- [2] X. Li et al. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670v4, 2026. <https://arxiv.org/abs/2602.12670v4>
- [3] Y. Li. Dynamic Agent Skills: A Lifecycle Survey and Taxonomy of Evolving Skill Libraries. arXiv:2607.10113v1, 2026. <https://arxiv.org/abs/2607.10113v1>
- [4] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291v2, 2023. <https://arxiv.org/abs/2305.16291v2>
- [5] N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366v4, 2023. <https://arxiv.org/abs/2303.11366v4>
- [6] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651v2, 2023. <https://arxiv.org/abs/2303.17651v2>
- [7] L. A. Agrawal et al. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv:2507.19457v2, 2025. <https://arxiv.org/abs/2507.19457v2>
- [8] S. Alzubi et al. EvoSkill: Automated Skill Discovery for Multi-Agent Systems. arXiv:2603.02766v1, 2026. <https://arxiv.org/abs/2603.02766v1>
- [9] P. Xia et al. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234v1, 2026. <https://arxiv.org/abs/2602.08234v1>
- [10] Y. Yang et al. AutoSkill: Experience-Driven Lifelong Learning via Skill Self-Evolution. arXiv:2603.01145v2, 2026. <https://arxiv.org/abs/2603.01145v2>
- [11] Q. Zhang et al. Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models. arXiv:2510.04618v3, 2025. <https://arxiv.org/abs/2510.04618v3>
- [12] H. Zhang et al. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. arXiv:2604.01687v3, 2026. <https://arxiv.org/abs/2604.01687v3>
- [13] X. Bai et al. SkillZip: Evaluation-Free Skill Compression for Self-Evolving Agents by Discovering Reusable Structure. arXiv:2608.11079v2, 2026. <https://arxiv.org/abs/2608.11079v2>
- [14] Y. Liu et al. Rethinking Self-Evolving Agent Skills: Feedback Dynamics over Multiple Rounds. arXiv:2608.02636v1, 2026. <https://arxiv.org/abs/2608.02636v1>
- [15] J. Zhu et al. SkillCoach: Self-Evolving Rubrics for Evaluating and Enhancing Agentic Skill-Use. arXiv:2607.01874v1, 2026. <https://arxiv.org/abs/2607.01874v1>
- [16] K. Ding et al. Agent Skill Evaluation and Evolution: Frameworks and Benchmarks. arXiv:2606.11435v1, 2026. <https://arxiv.org/abs/2606.11435v1>
- [17] D. Sculley et al. Hidden Technical Debt in Machine Learning Systems. NeurIPS, 2015, pp. 2503-2511. <https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf>
- [18] Anthropic. Demystifying evals for AI agents. Anthropic Engineering, 2026. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- [19] Anthropic. Effective context engineering for AI agents. Anthropic Engineering, 2025. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>